

File Organization & R Data Importing

PSYC 259: Principles of Data Science

John Franchak & Holly O'Rourke

Outline for Today

- File organization and naming conventions
 - Absolute and relative paths
- Importing CSV files
 - Importing single and multiple files
- Useful importing capabilities
- Other data import packages

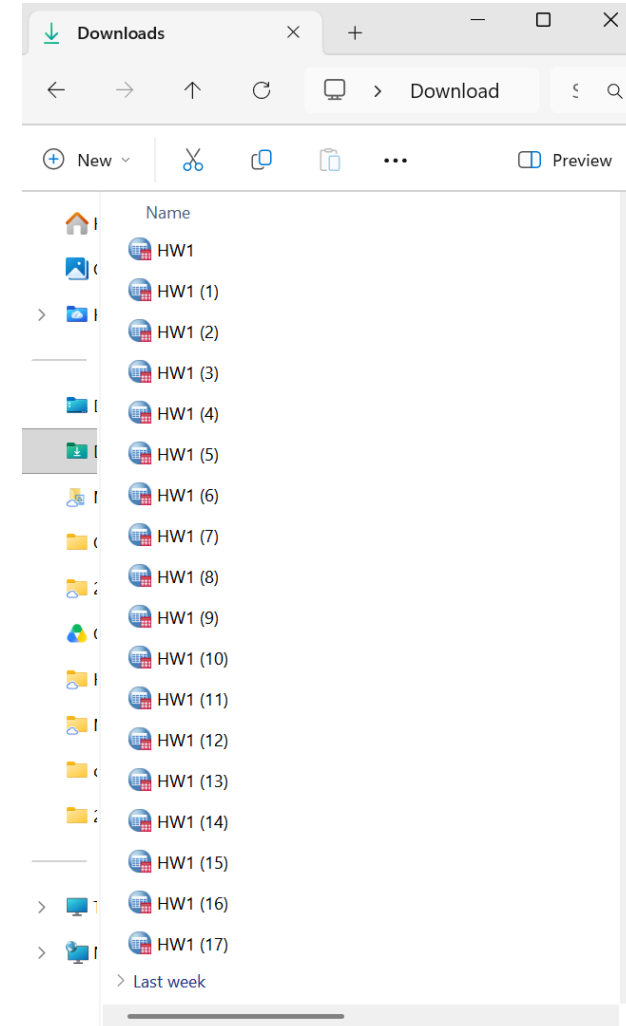
 Follow along from this week's Github repo

Last updated: 2026-10-09

Part 1: File Organization and Naming

File Organization: Why is it Important?

- How you structure the directories for a project
- Very often ignored, but essential for an efficient workflow
- Benefits:
 - Faster retrieval (prevents "losing" items)
 - Better data security (prevents accidental access or deletion)
 - Fewer errors (prevents duplicate work, accidental overwriting)

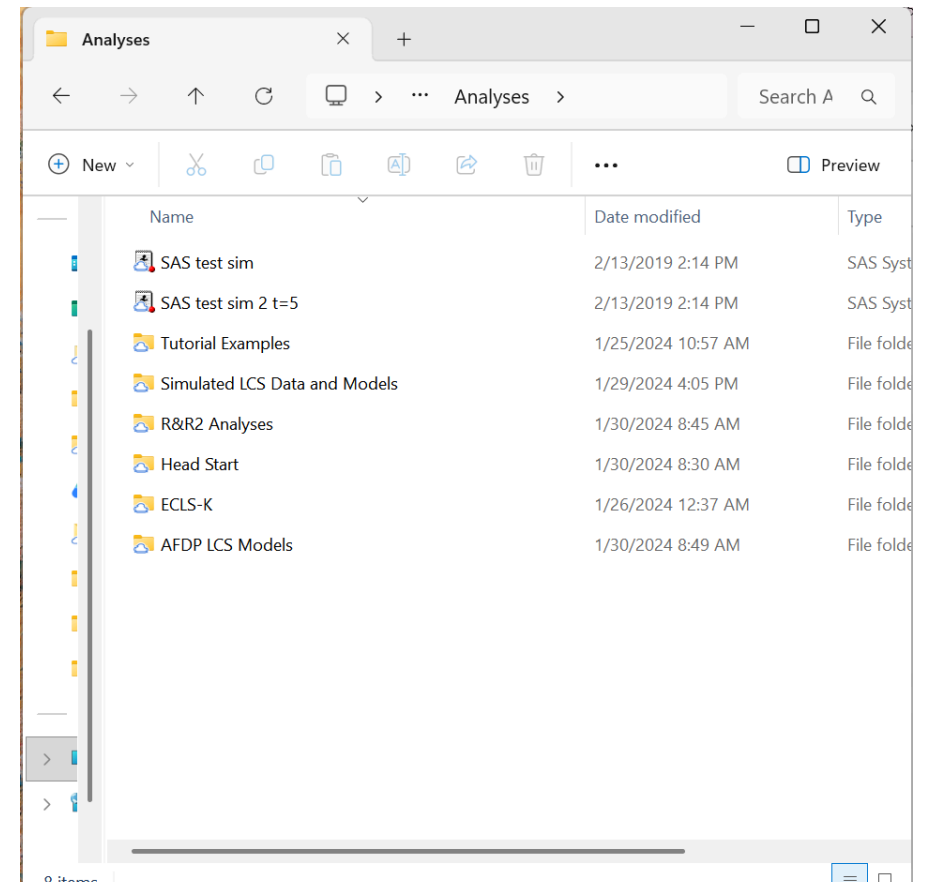
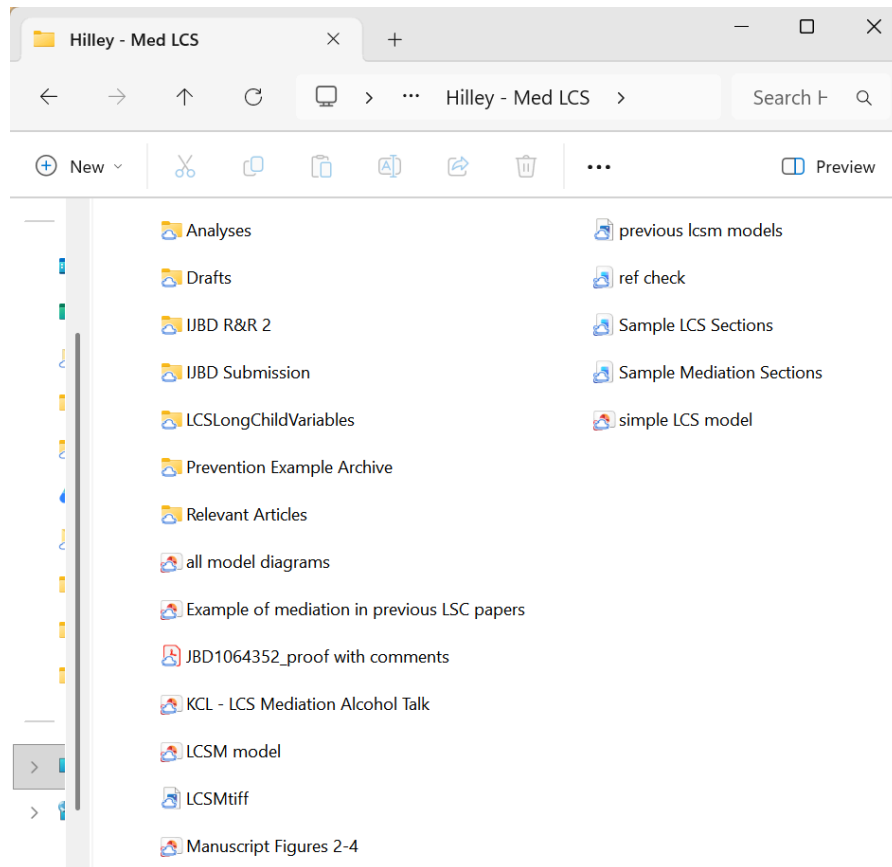


File Organization: Directories

- Directories: Where your files "live"
 - **Local**: They live on your machine
 - /Users/jonah/Desktop
 - **Cloud**: They live on a remote server hosted by a third party
 - /Users/jonah/Library/CloudStorage/GoogleDrive-jonahf@gmail.com
- Always maintain **local** directories when using git and Github
- We can use simple R code to create directories as well:
 - `if (!dir.exists("data_cleaned")) dir.create("data_cleaned")`

BAD File Organization

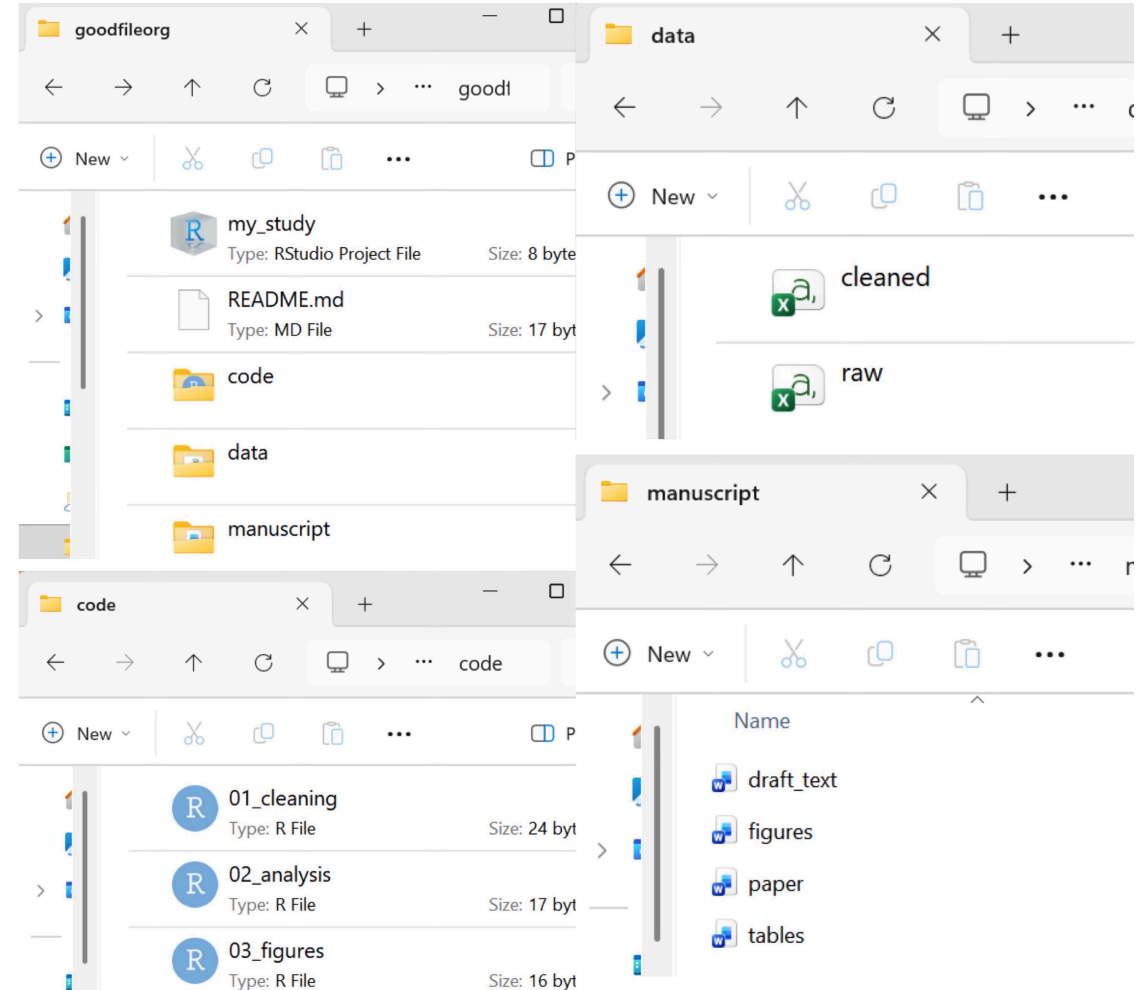
Why is this bad? Would you know where to find the data and analyses for the published paper?



GOOD File Organization

goodfileorg/

- README.md
- my_study.Rproj
- code/
 - 01_clean_data.R
 - 02_analysis.R
 - 03_figures.R
- data/
 - raw.csv
 - processed.csv
- manuscript/
 - draft_text.docx
 - figures.docx
 - tables.docx



Absolute vs. Relative File Paths

- This is a major consideration for our studies due to data privacy/IRB requirements
- Data are often stored with secure mapped drives that lead to different, machine-specific **absolute** file paths
 - Ex. Jonah's data is stored at
/Users/jonah/UniversityDrive/IRB_Study_123/participant_data.csv
 - Liz's data is stored at
/Users/lizdiaz/SecureResearch/IRB_Study_123/participant_data.csv
- You probably are familiar with using absolute file paths to import data in R
- With RStudio Projects, we can use only **relative** paths to ensure the code works for everyone on the project
 - Ex. data/participant_data.csv is the reference for everyone in the R code
 - More on this in a bit...

Naming Conventions

- How you name your files within a single subdirectory
- An ideal naming schema has three desirable properties:
 1. Machine readable
 2. Human readable
 3. Can be "default" sorted

Machine Readable

- Use Latin alphabet letters (abc), Arabic numerals (123), alphanumeric only (no special characters)
- Add leading zeros based on your expected count (ex. n = 200 -> 001, 002... 200)
- Spacing options:
 - Underscores (snake_case)
 - Capital letters (CamelCase)
 - Hyphens (kebab-case)
 - No spaces or special characters
- Choose case sensitivity wisely!
 - R is a case-sensitive language and uses snake_case for naming
- **Jonah's Data.csv** -> ?

Human Readable

- Consistency is key
 - Ex. Choose snake case and use it for ALL files in a project (scripts, data, etc.)
- File names should be concise, but detailed enough that another person can understand
 - Leave out unnecessary info like software
 - Ex. "Jonah's R Data Cleaning Script.R" -> "data_cleaning.R"
- Document your naming convention in a README file
 - Your future self and collaborators will thank you!

Default Sorting

- Before you start your workflow, decide on a sorting system that will make it easy for you to access your files with code
- Two typical sorting systems for prefixes:
 - Chronological
 - 2026-01-01_id001_task1.csv
 - 2026-01-01_id002_task1.csv
 - 2026-01-03_id001_task1.csv
 - Logical
 - 01_cleaning.R
 - 02_analysis.R
 - 03_figures.R
- You can also use version numbers as suffixes (when needed)
 - Ex. V01, V02, V03, etc. (instead of final, final_final, final_forreal)

Bad vs. Good Naming Conventions

```
print(bad_data)
```

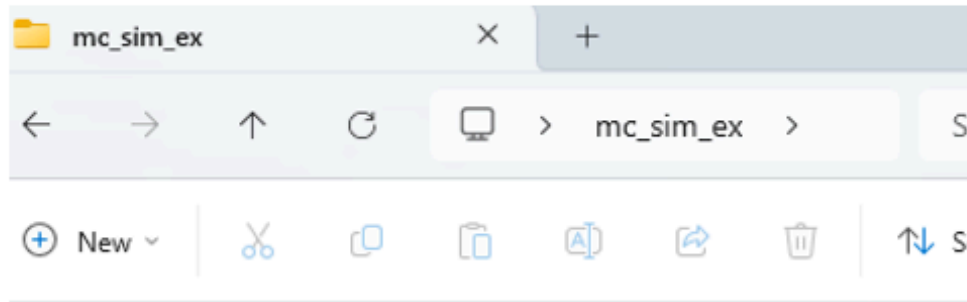
```
      x  y  a g t s1 s2
1 23 14 19 F 1  7 12
2 31 21 22 M 0  4 18
3 27 17 20 F 1  9 15
```

```
print(good_data)
```

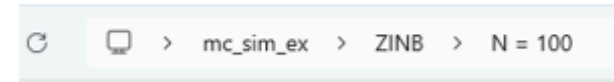
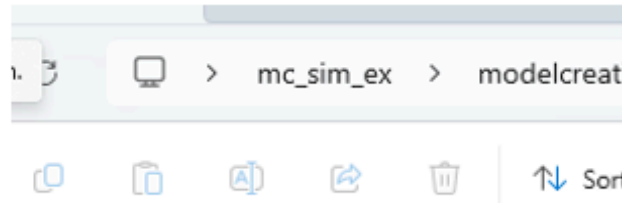
```
participant_age depression_score anxiety_score gender therapy_currently
1              23              14              19      F              1
2              31              21              22      M              0
3              27              17              20      F              1

pretest_score posttest_score
1              7              12
2              4              18
3              9              15
```

Complex Naming Schema Example



“modelcreation” folder
contains the model creation
scripts for each condition



Name	Type
------	------

ZINB_n100_1	INP File
ZINB_n100_2	INP File
ZINB_n100_3	INP File
ZINB_n100_4	INP File
ZINB_n100_5	INP File
ZINB_n100_6	INP File
ZINB_n100_7	INP File
ZINB_n100_8	INP File
ZINB_n100_9	INP File
ZINB_n100_10	INP File
ZINB_n100_11	INP File
ZINB_n100_12	INP File
ZINB_n100_13	INP
ZINB_n100_14	INP
ZINB_n100_15	INP
ZINB_n100_16	INP
ZINB_n100_17	INP
ZINB_n100_18	INP

Sub-directories
now contain
estimation scripts
for each
replication

Part 2: Importing Data

Where does R look for files?

- **Working directory:** The folder R is currently "working" from
- A **relative** path starts from the working directory
 - Ex. `data_raw/vocab16.csv`
- An **absolute** path starts from the root of the drive
 - Ex. `/Users/jonah/project/data_raw/vocab16.csv`

```
getwd()      # Where is R looking right now?  
list.files() # What files can R see from the getwd() directory?
```

- We need to make sure that the working directory contains the files for our current project

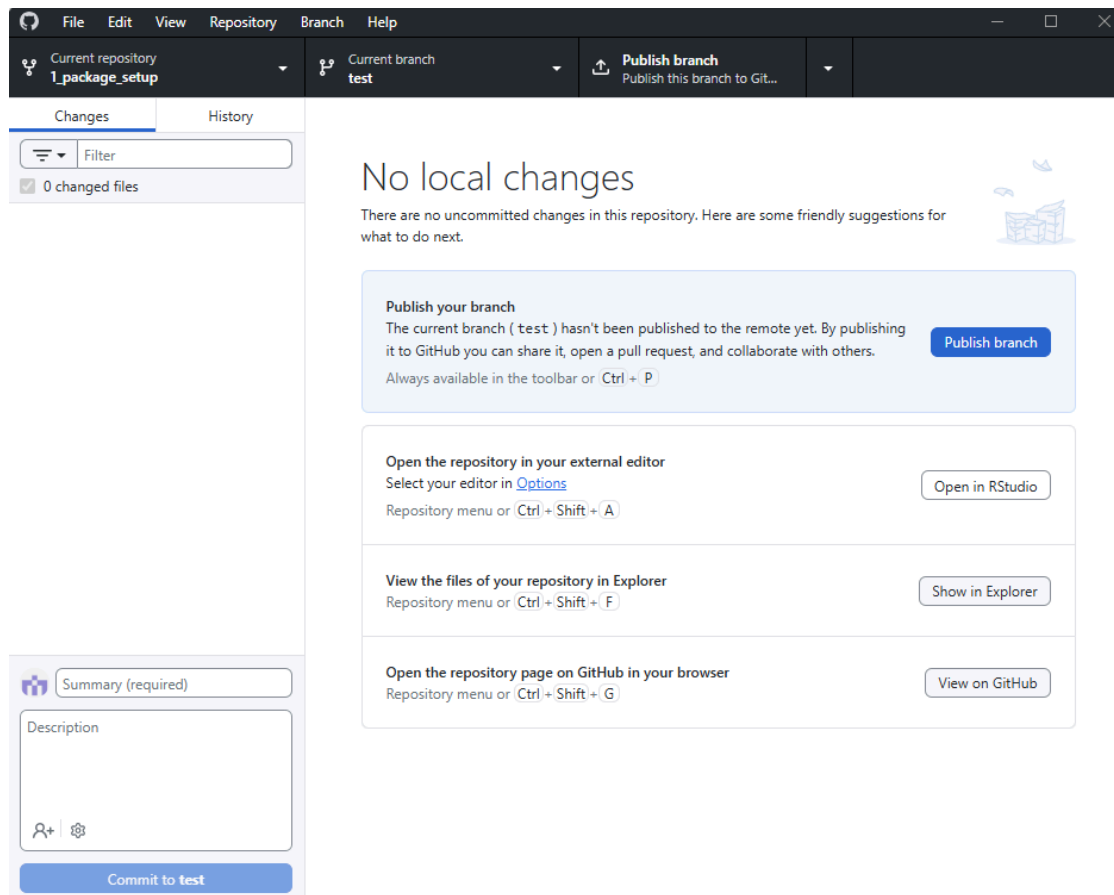
RStudio Projects set the working directory for you

- Opening the `.Rproj` file sets the working directory to the project folder
- Everyone who opens it gets the **same** starting point, so the same relative paths work on every machine
- Let's check this out with the repository we forked last week
 - The "1_package_setup" repo has both `.R` and `.Rproj` files

Quick Tutorial

Step 1: Open the project on your local machine

- Open GitHub desktop
- Select "1_package_setup" as current repository
- Select "Open in RStudio"



Step 2: Relative paths with the 'here' package

- `here()` finds the project root from the `.Rproj` file and builds paths from it
- Everyone writes the same line of code, `here()` makes it work on any machine
- Open a new R script to try it out
- Compare with your neighbor; what do you see?

```
# install.packages("here") if needed  
library(here)  
  
# the project root  
# absolute, differs per computer  
here()  
  
# root + file name  
here("package_install.Rproj")  
  
# TRUE from any folder in the project  
file.exists(here("package_install.Rproj"))
```

Step 2: Relative paths with the 'here' package

- `here()` finds the project root from the `.Rproj` file and builds paths from it
- Everyone writes the same line of code, `here()` makes it work on any machine
- Open a new R script to try it out
- Compare with your neighbor; what do you see?

```
# install.packages("here") if needed  
library(here)  
  
# the project root  
# absolute, differs per computer  
here()  
  
# root + file name  
here("package_install.Rproj")  
  
# TRUE from any folder in the project  
file.exists(here("package_install.Rproj"))
```

Different computers print different roots, but `here("package_install.Rproj")` is the same code for both

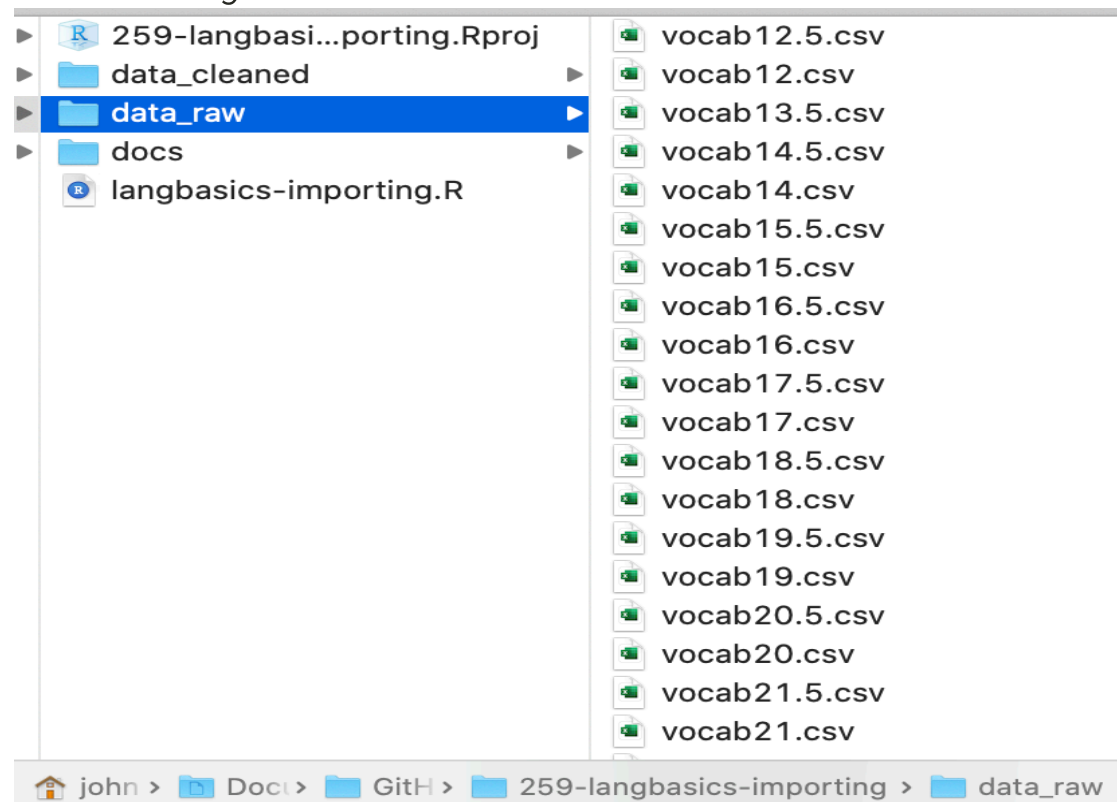
Data Importing: Base R and tidyverse

- Base R refers to the built-in functions in R
- tidyverse is a collection of packages that are specifically developed for data cleaning and visualization
 - Ex. dplyr, readr, ggplot2, tidyr, etc.
 - `install.packages("tidyverse")`
- Either method can be used for data cleaning and visualization, tidyverse can sometimes be more efficient

Importing a CSV File

- read.csv is base R, read_csv is tidyverse (readr package)
 - read_csv is faster
 - read_csv makes fewer assumptions about your data
 - read_csv can combine multiple data files into one

The following examples use .csv (comma separated value) files in the data_raw directory



```
# read.csv is part of base R, the default fx set  
ds <- read.csv('data_raw/vocab16.csv')
```



```
# read.csv is part of base R, the default fx set  
ds <- read.csv('data_raw/vocab16.csv')  
print(ds)
```

	age	word
1	16	shoes
2	16	berry
3	16	hi
4	16	diaper
5	16	teeth
6	16	uhoh

```
# read.csv is part of base R, the default fx set  
ds <- read.csv('data_raw/vocab16.csv')  
print(ds)
```

```
# Let's load the readr package to use read_csv
```

```
library(readr) #for read_csv
```

	age	word
1	16	shoes
2	16	berry
3	16	hi
4	16	diaper
5	16	teeth
6	16	uhoh

```
# read.csv is part of base R, the default fx set
ds <- read.csv('data_raw/vocab16.csv')
print(ds)

# Let's load the readr package to use read_csv
library(readr) #for read_csv
ds <- read_csv('data_raw/vocab16.csv')
```

	age	word
1	16	shoes
2	16	berry
3	16	hi
4	16	diaper
5	16	teeth
6	16	uhoh

```

# read.csv is part of base R, the default fx set
ds <- read.csv('data_raw/vocab16.csv')
print(ds)

# Let's load the readr package to use read_csv
library(readr) #for read_csv
ds <- read_csv('data_raw/vocab16.csv')
print(ds)

```

```

  age  word
1  16 shoes
2  16 berry
3  16   hi
4  16 diaper
5  16 teeth
6  16  uhoh

```

```
# A tibble: 6 × 2
```

```

  age word
<dbl> <chr>
1    16 shoes
2    16 berry
3    16 hi
4    16 diaper
5    16 teeth
6    16 uhoh

```

- "tibble" is a tidyverse data frame (keeps data frames predictable in tidyverse)

```
# We can use read_csv to read individual files  
ds12 <- read_csv('data_raw/vocab12.csv')
```

```
# We can use read_csv to read individual files  
ds12 <- read_csv('data_raw/vocab12.csv')  
print(ds12)
```

```
# A tibble: 3 × 2  
  age word  
  <dbl> <chr>  
1    12 book  
2    12 ball  
3    12 bye bye
```

```
# We can use read_csv to read individual files  
ds12 <- read_csv('data_raw/vocab12.csv')  
print(ds12)
```

```
ds12.5 <- read_csv('data_raw/vocab12.5.csv')
```

```
# A tibble: 3 × 2  
  age word  
  <dbl> <chr>  
1    12 book  
2    12 ball  
3    12 bye bye
```

```
# We can use read_csv to read individual files
ds12 <- read_csv('data_raw/vocab12.csv')
print(ds12)

ds12.5 <- read_csv('data_raw/vocab12.5.csv')
print(ds12.5)
```

```
# A tibble: 3 × 2
  age word
<dbl> <chr>
1    12 book
2    12 ball
3    12 bye bye
```

```
# A tibble: 1 × 2
  age word
<dbl> <chr>
1  12.5 dad
```



```
# We can use read_csv to read individual files
ds12 <- read_csv('data_raw/vocab12.csv')
print(ds12)

ds12.5 <- read_csv('data_raw/vocab12.5.csv')
print(ds12.5)

ds13.5 <- read_csv('data_raw/vocab13.5.csv')
```

```
# A tibble: 3 × 2
  age word
<dbl> <chr>
1    12 book
2    12 ball
3    12 bye bye

# A tibble: 1 × 2
  age word
<dbl> <chr>
1  12.5 dad
```

```
# We can use read_csv to read individual files
ds12 <- read_csv('data_raw/vocab12.csv')
print(ds12)

ds12.5 <- read_csv('data_raw/vocab12.5.csv')
print(ds12.5)

ds13.5 <- read_csv('data_raw/vocab13.5.csv')
print(ds13.5)
```

```
# A tibble: 3 × 2
  age word
<dbl> <chr>
1    12 book
2    12 ball
3    12 bye bye
```

```
# A tibble: 1 × 2
  age word
<dbl> <chr>
1  12.5 dad
```

```
# A tibble: 1 × 2
  age word
<dbl> <chr>
1  13.5 cat
```

```
# We can use read_csv to read individual files
ds12 <- read_csv('data_raw/vocab12.csv')
print(ds12)

ds12.5 <- read_csv('data_raw/vocab12.5.csv')
print(ds12.5)

ds13.5 <- read_csv('data_raw/vocab13.5.csv')
print(ds13.5)

# bind_rows can append tibbles together
# functionally merges the three data frames by column
ds_all <- bind_rows(ds12, ds12.5, ds13.5)
```

```
# A tibble: 3 × 2
  age word
<dbl> <chr>
1    12 book
2    12 ball
3    12 bye bye
```

```
# A tibble: 1 × 2
  age word
<dbl> <chr>
1  12.5 dad
```

```
# A tibble: 1 × 2
  age word
<dbl> <chr>
1  13.5 cat
```

```
# We can use read_csv to read individual files
ds12 <- read_csv('data_raw/vocab12.csv')
print(ds12)

ds12.5 <- read_csv('data_raw/vocab12.5.csv')
print(ds12.5)

ds13.5 <- read_csv('data_raw/vocab13.5.csv')
print(ds13.5)

# bind_rows can append tibbles together
# functionally merges the three data frames by column
ds_all <- bind_rows(ds12, ds12.5, ds13.5)
print(ds_all)
```

```
# A tibble: 3 × 2
  age word
<dbl> <chr>
1    12 book
2    12 ball
3    12 bye bye
```

```
# A tibble: 1 × 2
  age word
<dbl> <chr>
1  12.5 dad
```

```
# A tibble: 1 × 2
  age word
<dbl> <chr>
1  13.5 cat
```

```
# A tibble: 5 × 2
  age word
<dbl> <chr>
1    12 book
2    12 ball
3    12 bye bye
4  12.5 dad
5  13.5 cat
```

But imagine having to read in every file in the list with separate `read_csv` commands...

```
#function for listing files a directory  
list.files('data_raw', full.names = TRUE)
```

```
[1] "data_raw/vocab12.5.csv" "data_raw/vocab12.csv"  
[3] "data_raw/vocab13.5.csv" "data_raw/vocab14.5.csv"  
[5] "data_raw/vocab14.csv"   "data_raw/vocab15.5.csv"  
[7] "data_raw/vocab15.csv"   "data_raw/vocab16.5.csv"  
[9] "data_raw/vocab16.csv"   "data_raw/vocab17.5.csv"  
[11] "data_raw/vocab17.csv"   "data_raw/vocab18.5.csv"  
[13] "data_raw/vocab18.csv"   "data_raw/vocab19.5.csv"  
[15] "data_raw/vocab19.csv"   "data_raw/vocab20.5.csv"  
[17] "data_raw/vocab20.csv"   "data_raw/vocab21.5.csv"  
[19] "data_raw/vocab21.csv"   "data_raw/vocab22.5.csv"  
[21] "data_raw/vocab22.csv"   "data_raw/vocab23.5.csv"
```

```
# read_csv can import multiple files all at once!  
  
# Create object that stores all the file names and directories  
full_file_names <- list.files('data_raw', full.names = TRUE)
```

```
# read_csv can import multiple files all at once!  
  
# Create object that stores all the file names and directories  
full_file_names <- list.files('data_raw', full.names = TRUE)  
  
# Pass the names to read_csv to read all of them into a single tibble  
ds_all <- read_csv(full_file_names)
```

```
# read_csv can import multiple files all at once!

# Create object that stores all the file names and directories
full_file_names <- list.files('data_raw', full.names = TRUE)

# Pass the names to read_csv to read all of them into a single tibble
ds_all <- read_csv(full_file_names)
print(ds_all)
```

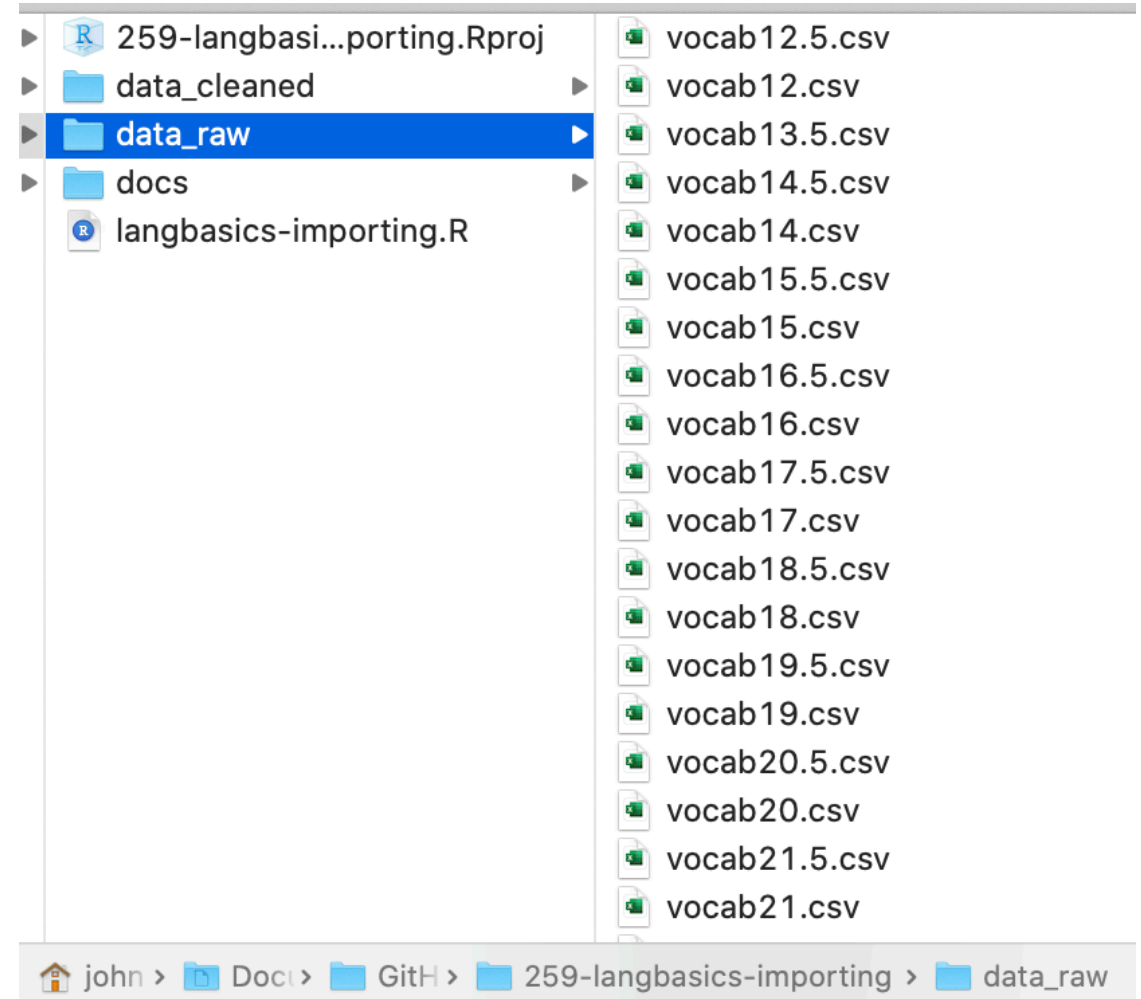
```
# A tibble: 440 × 2
```

```
  age word
<dbl> <chr>
1  12.5 dad
2   12  book
3   12  ball
4   12  bye bye
5  13.5 cat
6  14.5 socks/shoes
7  14.5 dog (animal)
8   14  baby
9  15.5 cheese
10 15.5 turkey
# i 430 more rows
```


Why did we need full.names?

- `list.files('data_raw')` gives `vocab12.csv`, `vocab12.5.csv`... which are only the file names
- `list.files('data_raw', full.names = TRUE)` gives the relative path including the directory:

`data_raw/vocab12.csv`,
`data_raw/vocab12.5.csv`



fs Package

- We can use the fs package to have more control over what we see of files
- `dir_ls()`: List files with location
- `path_file()`: Extract file names from a path string
- `path_ext_remove()`: Remove the final file extension from a filename
- `path_abs()`: Show absolute version of a given file path

```
# Use the fs package to have more control over files  
# install.packages("fs") if needed # install.packages("fs") if needed
```

```
library(fs)  
files <- dir_ls("data_raw")  
print(files)
```

```
data_raw/vocab12.5.csv data_raw/vocab12.csv data_raw/vocab13.5.csv  
data_raw/vocab14.5.csv data_raw/vocab14.csv data_raw/vocab15.5.csv  
data_raw/vocab15.csv data_raw/vocab16.5.csv data_raw/vocab16.csv  
data_raw/vocab17.5.csv data_raw/vocab17.csv data_raw/vocab18.5.csv  
data_raw/vocab18.csv data_raw/vocab19.5.csv data_raw/vocab19.csv  
data_raw/vocab20.5.csv data_raw/vocab20.csv data_raw/vocab21.5.csv  
data_raw/vocab21.csv data_raw/vocab22.5.csv data_raw/vocab22.csv  
data_raw/vocab23.5.csv data_raw/vocab23.csv data_raw/vocab24.csv
```

```
# Use the fs package to have more control over files  
# install.packages("fs") if needed # install.packages("fs") if needed
```

```
library(fs)  
files <- dir_ls("data_raw")  
print(path_file(files))
```

```
[1] "vocab12.5.csv" "vocab12.csv"   "vocab13.5.csv" "vocab14.5.csv"  
[5] "vocab14.csv"   "vocab15.5.csv" "vocab15.csv"   "vocab16.5.csv"  
[9] "vocab16.csv"   "vocab17.5.csv" "vocab17.csv"   "vocab18.5.csv"  
[13] "vocab18.csv"   "vocab19.5.csv" "vocab19.csv"   "vocab20.5.csv"  
[17] "vocab20.csv"   "vocab21.5.csv" "vocab21.csv"   "vocab22.5.csv"  
[21] "vocab22.csv"   "vocab23.5.csv" "vocab23.csv"   "vocab24.csv"
```

```
# Use the fs package to have more control over files  
# install.packages("fs") if needed # install.packages("fs") if needed
```

```
library(fs)  
files <- dir_ls("data_raw")  
print(path_ext_remove(path_file(files)))
```

```
[1] "vocab12.5" "vocab12"    "vocab13.5" "vocab14.5" "vocab14"    "vocab15.5"  
[7] "vocab15"   "vocab16.5" "vocab16"   "vocab17.5" "vocab17"    "vocab18.5"  
[13] "vocab18"   "vocab19.5" "vocab19"   "vocab20.5" "vocab20"    "vocab21.5"  
[19] "vocab21"   "vocab22.5" "vocab22"   "vocab23.5" "vocab23"    "vocab24"
```

```
# Use the fs package to have more control over files  
# install.packages("fs") if needed # install.packages("fs") if needed  
  
library(fs)  
files <- dir_ls("data_raw")  
print(path_abs(files))
```

```
C:/Users/holly/GitHub/datasci259/2-fileorg-importing/docs/data_raw/vocab12.5.csv  
C:/Users/holly/GitHub/datasci259/2-fileorg-importing/docs/data_raw/vocab12.csv  
C:/Users/holly/GitHub/datasci259/2-fileorg-importing/docs/data_raw/vocab13.5.csv  
C:/Users/holly/GitHub/datasci259/2-fileorg-importing/docs/data_raw/vocab14.5.csv  
C:/Users/holly/GitHub/datasci259/2-fileorg-importing/docs/data_raw/vocab14.csv  
C:/Users/holly/GitHub/datasci259/2-fileorg-importing/docs/data_raw/vocab15.5.csv  
C:/Users/holly/GitHub/datasci259/2-fileorg-importing/docs/data_raw/vocab15.csv  
C:/Users/holly/GitHub/datasci259/2-fileorg-importing/docs/data_raw/vocab16.5.csv  
C:/Users/holly/GitHub/datasci259/2-fileorg-importing/docs/data_raw/vocab16.csv  
C:/Users/holly/GitHub/datasci259/2-fileorg-importing/docs/data_raw/vocab17.5.csv  
C:/Users/holly/GitHub/datasci259/2-fileorg-importing/docs/data_raw/vocab17.csv  
C:/Users/holly/GitHub/datasci259/2-fileorg-importing/docs/data_raw/vocab18.5.csv  
C:/Users/holly/GitHub/datasci259/2-fileorg-importing/docs/data_raw/vocab18.csv  
C:/Users/holly/GitHub/datasci259/2-fileorg-importing/docs/data_raw/vocab19.5.csv  
C:/Users/holly/GitHub/datasci259/2-fileorg-importing/docs/data_raw/vocab19.csv  
C:/Users/holly/GitHub/datasci259/2-fileorg-importing/docs/data_raw/vocab20.5.csv  
C:/Users/holly/GitHub/datasci259/2-fileorg-importing/docs/data_raw/vocab20.csv  
C:/Users/holly/GitHub/datasci259/2-fileorg-importing/docs/data_raw/vocab21.5.csv  
C:/Users/holly/GitHub/datasci259/2-fileorg-importing/docs/data_raw/vocab21.csv  
C:/Users/holly/GitHub/datasci259/2-fileorg-importing/docs/data_raw/vocab22.5.csv  
C:/Users/holly/GitHub/datasci259/2-fileorg-importing/docs/data_raw/vocab22.csv  
C:/Users/holly/GitHub/datasci259/2-fileorg-importing/docs/data_raw/vocab23.5.csv
```

```
# Let's add some things and write a combined output
```

```
# Create a new column in a dataset
```

```
ds_all$ppt_name <- "Jonah"
```

```
# Let's add some things and write a combined output
```

```
# Create a new column in a dataset
```

```
ds_all$ppt_name <- "Jonah"
```

```
# Create a calculated column
```

```
ds_all$age_round <- round(ds_all$age)
```



```
# Let's add some things and write a combined output
```

```
# Create a new column in a dataset
```

```
ds_all$ppt_name <- "Jonah"
```

```
# Create a calculated column
```

```
ds_all$age_round <- round(ds_all$age)
```

```
# See the results
```

```
print(ds_all)
```

```
# A tibble: 440 × 4
```

	age	word	ppt_name	age_round
	<dbl>	<chr>	<chr>	<dbl>
1	12.5	dad	Jonah	12
2	12	book	Jonah	12
3	12	ball	Jonah	12
4	12	bye bye	Jonah	12
5	13.5	cat	Jonah	14
6	14.5	socks/shoes	Jonah	14
7	14.5	dog (animal)	Jonah	14
8	14	baby	Jonah	14
9	15.5	cheese	Jonah	16
10	15.5	turkey	Jonah	16

```
# i 430 more rows
```

```
# Let's add some things and write a combined output
```

```
# Create a new column in a dataset
```

```
ds_all$ppt_name <- "Jonah"
```

```
# Create a calculated column
```

```
ds_all$age_round <- round(ds_all$age)
```

```
# See the results
```

```
print(ds_all)
```

```
# Create the data_cleaned directory if it doesn't exist yet
```

```
if (!dir.exists("data_cleaned")) dir.create("data_cleaned")
```

```
# A tibble: 440 × 4
```

	age	word	ppt_name	age_round
	<dbl>	<chr>	<chr>	<dbl>
1	12.5	dad	Jonah	12
2	12	book	Jonah	12
3	12	ball	Jonah	12
4	12	bye bye	Jonah	12
5	13.5	cat	Jonah	14
6	14.5	socks/shoes	Jonah	14
7	14.5	dog (animal)	Jonah	14
8	14	baby	Jonah	14
9	15.5	cheese	Jonah	16
10	15.5	turkey	Jonah	16

```
# i 430 more rows
```

```

# Let's add some things and write a combined output

# Create a new column in a dataset
ds_all$ppt_name <- "Jonah"

# Create a calculated column
ds_all$age_round <- round(ds_all$age)

# See the results
print(ds_all)

# Create the data_cleaned directory if it doesn't exist yet
if (!dir.exists("data_cleaned")) dir.create("data_cleaned")

# Let's export the combined data as a new dataset

write_csv(ds_all, file = "data_cleaned/vocab_combined.csv")

```

```

# A tibble: 440 × 4
   age word      ppt_name age_round
  <dbl> <chr>    <chr>      <dbl>
1  12.5 dad      Jonah        12
2   12  book      Jonah        12
3   12  ball      Jonah        12
4   12  bye bye    Jonah        12
5  13.5 cat      Jonah        14
6  14.5 socks/shoes Jonah        14
7  14.5 dog (animal) Jonah        14
8   14  baby      Jonah        14
9  15.5 cheese    Jonah        16
10  15.5 turkey    Jonah        16
# i 430 more rows

```

Useful readr read_csv() capabilities

- Important read_csv() arguments:
 - `col_names = TRUE` (reads column names from first line by default)
 - `col_names = FALSE` (treats the first line as data)
 - `col_names = c("col1name", "col2name")` (to specify the names)
 - `col_types = NULL` (by default, guesses the data type)
 - `col_types = "ccDin"` (specify types character/date/integer/number)
 - `skip = 10` (skip the first 10 lines)
- Can also be used with `read_tsv()` (tab-delimited), `read_delim()` (character-delimited, general)
- `write_csv()`, `write_tsv()`, `write_delim()` export/save data

```
fname <- "data_cleaned/vocab_combined.csv"
colname <- c("AGE", "WORD", "NAME", "MONTH")
coltypes <- "cccc"
ds <- read_csv(file = fname)
print(ds)
```

```
# A tibble: 440 × 4
```

	age	word	ppt_name	age_round
	<dbl>	<chr>	<chr>	<dbl>
1	12.5	dad	Jonah	12
2	12	book	Jonah	12
3	12	ball	Jonah	12
4	12	bye bye	Jonah	12
5	13.5	cat	Jonah	14
6	14.5	socks/shoes	Jonah	14
7	14.5	dog (animal)	Jonah	14
8	14	baby	Jonah	14
9	15.5	cheese	Jonah	16
10	15.5	turkey	Jonah	16

```
# i 430 more rows
```

```
fname <- "data_cleaned/vocab_combined.csv"
colname <- c("AGE", "WORD", "NAME", "MONTH")
coltypes <- "cccc"
ds <- read_csv(file = fname, col_names = FALSE)
print(ds)
```

```
# A tibble: 441 × 4
```

	X1	X2	X3	X4
	<chr>	<chr>	<chr>	<chr>
1	age	word	ppt_name	age_round
2	12.5	dad	Jonah	12
3	12	book	Jonah	12
4	12	ball	Jonah	12
5	12	bye bye	Jonah	12
6	13.5	cat	Jonah	14
7	14.5	socks/shoes	Jonah	14
8	14.5	dog (animal)	Jonah	14
9	14	baby	Jonah	14
10	15.5	cheese	Jonah	16

```
# i 431 more rows
```

```
fname <- "data_cleaned/vocab_combined.csv"
colname <- c("AGE", "WORD", "NAME", "MONTH")
coltypes <- "cccc"
ds <- read_csv(file = fname, col_names = colname)
print(ds)
```

```
# A tibble: 441 × 4
```

	AGE	WORD	NAME	MONTH
	<chr>	<chr>	<chr>	<chr>
1	age	word	ppt_name	age_round
2	12.5	dad	Jonah	12
3	12	book	Jonah	12
4	12	ball	Jonah	12
5	12	bye bye	Jonah	12
6	13.5	cat	Jonah	14
7	14.5	socks/shoes	Jonah	14
8	14.5	dog (animal)	Jonah	14
9	14	baby	Jonah	14
10	15.5	cheese	Jonah	16

```
# i 431 more rows
```

```
fname <- "data_cleaned/vocab_combined.csv"
colname <- c("AGE", "WORD", "NAME", "MONTH")
coltypes <- "cccc"
ds <- read_csv(file = fname, col_names = colname, skip = 1)
print(ds)
```

```
# A tibble: 440 × 4
```

	AGE	WORD	NAME	MONTH
	<dbl>	<chr>	<chr>	<dbl>
1	12.5	dad	Jonah	12
2	12	book	Jonah	12
3	12	ball	Jonah	12
4	12	bye bye	Jonah	12
5	13.5	cat	Jonah	14
6	14.5	socks/shoes	Jonah	14
7	14.5	dog (animal)	Jonah	14
8	14	baby	Jonah	14
9	15.5	cheese	Jonah	16
10	15.5	turkey	Jonah	16

```
# i 430 more rows
```



```
fname <- "data_cleaned/vocab_combined.csv"
colname <- c("AGE", "WORD", "NAME", "MONTH")
coltypes <- "cccc"
ds <- read_csv(file = fname, col_names = colname, skip = 1, col_types = coltypes)
print(ds)
```

```
# A tibble: 440 × 4
```

	AGE	WORD	NAME	MONTH
	<chr>	<chr>	<chr>	<chr>
1	12.5	dad	Jonah	12
2	12	book	Jonah	12
3	12	ball	Jonah	12
4	12	bye bye	Jonah	12
5	13.5	cat	Jonah	14
6	14.5	socks/shoes	Jonah	14
7	14.5	dog (animal)	Jonah	14
8	14	baby	Jonah	14
9	15.5	cheese	Jonah	16
10	15.5	turkey	Jonah	16

```
# i 430 more rows
```

More Resources

- tidyverse data import "cheatsheets"
- Read the documentation: `?read_csv`, `?write_csv`
- specialized import packages
 - `haven` for SPSS/Stata/SAS
 - `readxl` for .xlsx
 - `googlesheets4` for Google sheets

Data import with the tidyverse : : CHEAT SHEET



Read Tabular Data with readr

`read_*`(file, col_names = TRUE, col_types = NULL, col_select = NULL, id = NULL, locale, n_max = Inf, skip = 0, na = c("", "NA"), guess_max = min(1000, n_max), show_col_types = TRUE) See ?`read_delim`

A|B|C
1|2|3
4|5|NA

A	B	C
1	2	3
4	5	NA

read_delim("file.txt", delim = "|") Read files with any delimiter. If no delimiter is specified, it will automatically guess.
To make file.txt, run: `write_file("A|B|C\n1|2|3\n4|5|NA", file = "file.txt")`

A,B,C
1,2,3
4,5,NA

A	B	C
1	2	3
4	5	NA

read_csv("file.csv") Read a comma delimited file with period decimal marks.
`write_file("A,B,C\n1,2,3\n4,5,NA", file = "file.csv")`

A;B;C
1;5;2;3
4;5;5;NA

A	B	C
1.5	2	3
4.5	5	NA

read_csv2("file2.csv") Read semicolon delimited files with comma decimal marks.
`write_file("A;B;C\n1;5;2;3\n4;5;5;NA", file = "file2.csv")`

A B C
1 2 3
4 5 NA

A	B	C
1	2	3
4	5	NA

read_tsv("file.tsv") Read a tab delimited file. Also **read_table()**.
read_fwf("file.tsv", fwf_widths(c(2, 2, NA))) Read a fixed width file.
`write_file("A\tB\tC\n1\t2\t3\n4\t5\tNA\n", file = "file.tsv")`

USEFUL READ ARGUMENTS

A	B	C
1	2	3
4	5	NA

No header
`read_csv("file.csv", col_names = FALSE)`

x	y	z
1	2	3
4	5	NA

Provide header
`read_csv("file.csv",
col_names = c("x", "y", "z"))`

A B C
1 2 3
4 5 NA

Read multiple files into a single table
`read_csv(c("f1.csv", "f2.csv", "f3.csv"),
id = "origin_file")`

1	2	3
4	5	NA

Skip lines
`read_csv("file.csv", skip = 1)`

A	B	C
1	2	3

Read a subset of lines
`read_csv("file.csv", n_max = 1)`

A	B	C
NA	2	3
4	5	NA

Read values as missing
`read_csv("file.csv", na = c("1"))`

A;B;C
1;5;2;3;0

Specify decimal marks
`read_delim("file2.csv", locale =
locale(decimal_mark = ","))`

One of the first steps of a project is to import outside data into R. Data is often stored in tabular formats, like csv files or spreadsheets.



The front page of this sheet shows how to import and save text files into R using **readr**.



The back page shows how to import spreadsheet data from Excel files using **readxl** or Google Sheets using **googlesheets4**.

OTHER TYPES OF DATA

Try one of the following packages to import other types of files:

- **haven** - SPSS, Stata, and SAS files
- **DBI** - databases
- **jsonlite** - json
- **xml2** - XML
- **httr** - Web APIs
- **rvest** - HTML (Web Scraping)
- **readr::read_lines()** - text data

Column Specification with readr

Column specifications define what data type each column of a file will be imported as. By default readr will generate a column spec when a file is read and output a summary.

spec(x) Extract the full column specification for the given imported data frame.

```
spec(x)
# cols(
#   age = col_integer(),
#   sex = col_character(),
#   earn = col_double()
# )
```

age is an integer
sex is a character
earn is a double (numeric)

COLUMN TYPES

Each column type has a function and corresponding string abbreviation.

- **col_logical()** - "l"
- **col_integer()** - "i"
- **col_double()** - "d"
- **col_number()** - "n"
- **col_character()** - "c"
- **col_factor**(levels, ordered = FALSE) - "f"
- **col_datetime**(format = "") - "T"
- **col_date**(format = "") - "D"
- **col_time**(format = "") - "t"
- **col_skip()** - "s", "S"
- **col_guess()** - "?"

USEFUL COLUMN ARGUMENTS

Hide col spec message

`read_*(file, show_col_types = FALSE)`

Select columns to import

Use names, position, or selection helpers.
`read_*(file, col_select = c(age, earn))`

Guess column types

To guess a column type, `read_*`() looks at the first 1000 rows of data. Increase with **guess_max**.
`read_*(file, guess_max = Inf)`

DEFINE COLUMN SPECIFICATION

Set a default type

```
read_csv(
  file,
  col_type = list(default = col_double())
)
```

Use column type or string abbreviation

```
read_csv(
  file,
  col_type = list(x = col_double(), y = "l", z = "_")
)
```

Use a single string of abbreviations

```
# col types: skip, guess, integer, logical, character
read_csv(
  file,
  col_type = "_?ilc"
)
```

Import Spreadsheets

with readxl

READ EXCEL FILES

	A	B	C	D	E
1	x1	x2	x3	x4	x5
2	x		z	8	NA
3	y	7		9	10

s1

read_excel(path, sheet = NULL, range = NULL)
Read a .xls or .xlsx file based on the file extension. See front page for more read arguments. Also **read_xls()** and **read_xlsx()**.
`read_excel("excel_file.xlsx")`

READ SHEETS

A	B	C	D	E
s1	s2	s3		

read_excel(path, sheet = NULL) Specify which sheet to read by position or name.
`read_excel(path, sheet = 1)`
`read_excel(path, sheet = "s1")`

s1	s2	s3
----	----	----

excel_sheets(path) Get a vector of sheet names.
`excel_sheets("excel_file.xlsx")`

A	B	C	D	E
s1				
	s2	s3		

To read multiple sheets:

1. Get a vector of sheet names from the file path.
2. Set the vector names to be the sheet names.
3. Use `purrr::map_dfr()` to read multiple files into one data frame.

```
path <- "your_file_path.xlsx"
path %>% excel_sheets() %>%
  set_names() %>%
  map_dfr(read_excel, path = path)
```

OTHER USEFUL EXCEL PACKAGES

For functions to write data to Excel files, see:

- **openxlsx**
- **writexl**

For working with non-tabular Excel data, see:

- **tidyxl**



with googlesheets4

READ SHEETS

	A	B	C	D	E
1	x1	x2	x3	x4	x5
2	x		z	8	NA
3	y	7		9	10

s1

read_sheet(ss, sheet = NULL, range = NULL)
Read a sheet from a URL, a Sheet ID, or a dribble from the googledrive package. See front page for more read arguments. Same as **range_read()**.

SHEETS METADATA

URLs are in the form:
`https://docs.google.com/spreadsheets/d/SPREADSHEET_ID/edit#gid=SHEET_ID`

gs4_get(ss) Get spreadsheet meta data.

gs4_find(...) Get data on all spreadsheet files.

sheet_properties(ss) Get a tibble of properties for each worksheet. Also **sheet_names()**.

WRITE SHEETS

1	x	4
2	y	5
3	z	6

s1

write_sheet(data, ss = NULL, sheet = NULL)
Write a data frame into a new or existing Sheet.

gs4_create(name, ..., sheets = NULL) Create a new Sheet with a vector of names, a data frame, or a (named) list of data frames.

sheet_append(ss, data, sheet = 1) Add rows to the end of a worksheet.

A	B	C	D
1			
2			

s1

x1	x2	x3
1	x1	x2 x3
2	y	5
3	z	6

s1

READXL COLUMN SPECIFICATION

Column specifications define what data type each column of a file will be imported as.

Use the **col_types** argument of **read_excel()** to set the column specification.

Guess column types

To guess a column type, **read_excel()** looks at the first 1000 rows of data. Increase with the **guess_max** argument.

`read_excel(path, guess_max = Inf)`

Set all columns to same type, e.g. character

`read_excel(path, col_types = "text")`

Set each column individually

```
read_excel(
  path,
  col_types = c("text", "guess", "guess", "numeric")
)
```

COLUMN TYPES

logical	numeric	text	date	list
TRUE	2	hello	1947-01-08	hello
FALSE	3.45	world	1956-10-21	1

- skip
- guess
- logical
- numeric
- date
- list
- text

Use **list** for columns that include multiple data types. See **tidyr** and **purrr** for list-column data.

CELL SPECIFICATION FOR READXL AND GOOGLESHEETS4

A	B	C	D	E
1	1	2	3	4
2	x		y	z
3	6	7		9 10

s1

Use the **range** argument of **readxl::read_excel()** or **googlesheets4::read_sheet()** to read a subset of cells from a sheet.

```
read_excel(path, range = "Sheet1!B1:D2")
read_sheet(ss, range = "B1:D2")
```

Also use the range argument with cell specification functions **cell_limits()**, **cell_rows()**, **cell_cols()**, and **anchored()**.



GOOGLESHEETS4 COLUMN SPECIFICATION

Column specifications define what data type each column of a file will be imported as.

Use the **col_types** argument of **read_sheet()** or **range_read()** to set the column specification.

Guess column types

To guess a column type **read_sheet()** or **range_read()** looks at the first 1000 rows of data. Increase with **guess_max**.

`read_sheet(path, guess_max = Inf)`

Set all columns to same type, e.g. character

`read_sheet(path, col_types = "c")`

Set each column individually

col types: skip, guess, integer, logical, character
`read_sheets(ss, col_types = "?ilc")`

COLUMN TYPES

l	n	c	D	L
TRUE	2	hello	1947-01-08	hello
FALSE	3.45	world	1956-10-21	1

- skip - "-" or "-."
- guess - "?"
- logical - "l"
- integer - "i"
- double - "d"
- numeric - "n"
- date - "D"
- datetime - "T"
- character - "c"
- list-column - "L"
- cell - "C" Returns list of raw cell data.

Use **list** for columns that include multiple data types. See **tidyr** and **purrr** for list-column data.

FILE LEVEL OPERATIONS

googlesheets4 also offers ways to modify other aspects of Sheets (e.g. freeze rows, set column width, manage (work)sheets). Go to **googlesheets4.tidyverse.org** to read more.

For whole-file operations (e.g. renaming, sharing, placing within a folder), see the tidyverse package **googledrive** at **googledrive.tidyverse.org**.

Data Import Homework Assignment

- Use the GitHub repo link below to clone the project (no need to fork) to your own user account and then to work locally on your own computer.
- Be sure you are using R version ≥ 4.0 and readr version ≥ 2.0
- The homework will make sure you learned what we covered, and will also ask you to try out new things to extend your knowledge
- Push your answers to a public Github repo, and turn in the homework by entering the link to the repo on Canvas

 Homework Github repo